CMSC 25750: Large Language Models
Homework 1: LLM Interpretability

Instructor: Chenhao Tan

Due: Apr 1, 2026, 11:59 pm

This homework covers probing, the logit lens, attention mechanics, and KV-cache implementation — problems representative of research and industry interviews.

Submission Export your `hw1.ipynb` to a PDF file, and then upload to Gradescope with all cell outputs visible. This PDF is a read-only problem sheet; do **not** submit it.

Repository layout

```
HW1/
|-- hw1.ipynb              (submit this; run all cells, write answers here)
|-- hw1.pdf                (this problem sheet, read only)
|-- compile.py             (run to generate cache/ files)
|-- src/
|   |-- probing.py         (implement Section 1)
|   |-- logit_lens.py      (implement Section 2)
|   |-- kv_cache.py        (implement Section 4)
|-- tests/
|   |-- test_probing.py    (autograder; do not modify)
|   |-- test_kv_cache.py   (autograder; do not modify)
|-- cache/                 (auto-generated by compile.py)
|-- pyproject.toml
```

General Instructions See `README.md` for environment setup. You can work one section at a time: implement, test, compile, then fill in the notebook — without waiting to finish everything first.

1. Implement the coding problems in `src/probing.py`, `src/logit_lens.py`, and `src/kv_cache.py`.
2. Run the autograder for the completed section:

   ```
   pytest tests/test_probing.py -v    # after Section 1
   pytest tests/test_kv_cache.py -v   # after Section 4
   ```

3. Compile to generate cache files (requires GPU; see `README.md` for SLURM instructions):

   ```
   python compile.py section1   # or section2 / section4 / all
   ```

4. Run the corresponding cells in `hw1.ipynb` and write your answers in the designated markdown cells.

5. When all sections are done, export `hw1.ipynb` as PDF and submit to Gradescope.

Acknowledgements
This assignment is partially adapted from Understanding Language Models (Franke, Eickhoff, Tsvilodub).

Section 1: Linear Probing (25 pts)

When a transformer processes a sentence, each layer produces a hidden representation (a vector) for every token. These vectors encode information that the model has extracted from the input, but it is not obvious what information ends up where. *Probing* is a diagnostic technique for answering this question (Belinkov & Glass, 2019). A small, separate classifier (i.e., the **probe**) is trained to predict certain properties directly from the hidden representation at a particular layer.

Using a **linear** probe is deliberate: a linear classifier can only succeed if the target property is *linearly separable* in representation space. This keeps the probe weak enough that any success is attributable to structure in the representation, not to the probe's own expressivity (see Hewitt & Liang, 2019 for a careful treatment of this point).

This task. You will train a linear probe on hidden states of pythia-160m (a 160M-parameter causal language model) to test whether the model encodes **subject-verb number agreement** (e.g.... ***are*** vs. ... ***is***). The dataset is BLiMP (Warstadt et al., 2020), which pairs grammatical and ungrammatical sentences that differ only in the target verb form. You will compare probes trained on the **final layer** and the **third layer** to see how the model's syntactic knowledge evolves with depth.

Probe parameters.

- **Architecture**: a single `nn.Linear` layer. pythia-160m has hidden dimension $d = 512$, so the weight matrix is $W \in \mathbb{R}^{2\times 512}$ (2 output classes: grammatical / ungrammatical).
- **Input**: the mean-pooled hidden-state vector of the sentence at a chosen layer — a vector of size 512.
- **Loss**: cross-entropy.
- **Optimizer**: Adam, learning rate 10^{-3}, for 100 epochs.

Coding Problems

1.1: Implement `ProbingClassifier` **(4 pts)**

In `src/probing.py`, implement the `ProbingClassifier` class:

- **(a)** `__init__`: store a single `nn.Linear(input_dim, num_classes)` as `self.linear`.
- **(b)** `forward`: pass `x` through `self.linear` and return raw logits — no softmax.

1.2: Implement `build_classifier` **(4 pts)**

In `src/probing.py`, instantiate a `ProbingClassifier`, a `nn.CrossEntropyLoss`, and a `torch.optim.Adam` optimizer (lr $= 10^{-3}$); move the classifier to `device`; return all three.

1.3: Implement `get_sentence_repr` **(4 pts)**

In `src/probing.py`, tokenize a sentence, run a forward pass, extract the `hidden_states` tuple from the model output, and return a numpy array of shape `(num_layers + 1, seq_len, hidden_size)`.

1.4 Analysis

(a). **[4 pts]** State the test accuracy for both layers and the gap between them. What does this suggest about where syntactic information is stored in a transformer?

Deliverable: 2–3 sentences; cite specific numbers.

(b). **[4 pts]** Why is a linear probe preferred over a more powerful classifier (e.g. an MLP) when the goal is to interpret model representations?

Deliverable: 2–3 sentences.

(c). **[5 pts]** Suppose you train the same linear probe on hidden states from a *randomly initialized* (untrained) model of the same architecture and obtain high test accuracy. Can you conclude whether the trained model's representations encode subject-verb agreement? Why or why not?

Deliverable: 2–3 sentences.

Section 2: Logit Lens (20 pts)

At the end of a transformer, the residual stream is projected through an *unembedding matrix* to produce logits over the vocabulary. The **logit lens** (nostalgebraist, 2020) applies this same projection to the residual stream at *every intermediate layer*, turning each layer's hidden state into a vocabulary distribution. This lets us watch how the model's "best guess" for the next token evolves with depth — a lightweight window into the residual stream without modifying the model.

Read Olsson et al., 2022 before this section; the writing questions draw on its account of induction heads. Implement Problems 2.1 and 2.2 in `src/logit_lens.py`.

2.1: Implement `get_logit_lens_predictions` (6 pts)

Use nnsight tracing to intercept the residual stream at every layer, project through the final layer norm and unembedding, and return the full vocabulary distribution alongside top-1 summaries.
Deliverable: complete the code and then compile the notebook

2.2: Implement `get_token_rank_by_layer` (4 pts)

Given the full probability tensor and a target token ID, return the 1-indexed rank of that token at the last position for each layer.
Deliverable: complete the code and then compile the notebook

2.3 Additional Questions

(a). **[5 pts]** Suppose you observe that correct token reaching rank 1 at some layer $\ell^* < L$. Is it correct to say the model "decided" the answer at ℓ^*?

Deliverable: 2–3 sentences.

(b). **[5 pts]** According to Olsson et al., 2022, explain why in-context examples cause the correct token to rank highly at earlier layers.

Deliverable: 2–3 sentences.

Section 3: Attention Mechanics (30 pts)

Please show every intermediate step; partial credit is given for correct reasoning with arithmetic errors. Answer in the designated markdown cells in `hw1.ipynb`.

3.1 QKV Attention by Hand (10 pts)

Recall the scaled dot-product attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Here T is the sequence length (number of input tokens), d_k is the key/query dimension for this head, and $Q, K, V \in \mathbb{R}^{T \times d_k}$. Row i of Q is the *query* vector for token i; row j of K is the *key* vector for token j. The (i, j) entry of $QK^\top$ therefore measures how much token i attends to token j. The scaling factor $1/\sqrt{d_k}$ prevents dot products from growing large as d_k increases, which would push softmax into near-zero-gradient regions.

Consider when we have an autoregressive *decoder-only* model, with $T = 3$ tokens and key/query dimension $d_k = 3$. The query, key, and value matrices are:

$$Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$

Compute the attention calculation step by step.

3.2 FLOPs in Attention (10 pts)

A *FLOP* (floating-point operation) counts a single arithmetic operation (one multiply *or* one add). A *multiply-add* (fused multiply-add, or FMA) performs both at once and therefore counts as **2 FLOPs**. Consequently, one $m \times n$ matrix multiplied by an $n \times p$ matrix costs $m \cdot n \cdot p$ multiply-adds $= 2mnp$ FLOPs.

Understanding how FLOPs scale with sequence length T is practically important: for short sequences the cost is dominated by the weight projections ($\propto T \cdot d^2$), but for very long sequences the attention scores and weighted sums ($\propto T^2 \cdot d$) can become the bottleneck.

Use the following model configuration throughout:

- $d = 512$ — hidden (model) dimension
- $H = 8$ — number of attention heads; per-head dimension $d_h = d/H = 64$
- $T = 256$ — sequence length (number of tokens)
- $L = 6$ — number of transformer layers

In each attention layer the projections $W_Q, W_K, W_V, W_O \in \mathbb{R}^{d \times d}$ map the full hidden dimension to the full hidden dimension (the multi-head split happens logically after $W_Q/W_K/W_V$).

(a). **[6 pts]** Derive the total FLOPs for a single forward pass of single-head attention with batch size B, sequence length T, and model dimension d. Work through each component in order: the Q, K, V projections (each $W \in \mathbb{R}^{d \times d}$), the attention scores $QK^\top$, the weighted sum AV, and the output projection W_O. Sum them into a single formula in B, T, d, then calculate with $B = 4$ and the configuration above.

Deliverable: A few lines of computation.

(b). **[4 pts]** From your formula, at what sequence length T does the quadratic term begin to dominate the linear term? Express the threshold in terms of d and evaluate for $d = 512$.

Deliverable: A few lines of computation.

3.3 KV-Cache: Memory and FLOPs (10 pts)

In autoregressive generation, every new token must attend to all previous tokens. Without optimization, each decode step would re-run the full forward pass on the *entire* growing sequence, recomputing K and V for all prior tokens from scratch. A **KV-cache** stores the key and value tensors for previously processed tokens so that each decode step only needs to (i) project the one new token through W_Q, W_K, W_V, and (ii) attend to the cached keys and values without any recomputation. Each layer maintains its own (K, V) tensors of shape `(batch, num_heads, seq_len, d_head)`. KV-cache effectively reduces time complexity from $O(T^2)$ to $O(T)$ per step. If you are interested of how KV-cache works or need more background information, you can refer to this blog.

For the following questions, use $B = 4$, $d = 512$, $L = 6$, float16 ($b = 2$ bytes/element).

(a). **[5 pts]** Derive the formula for KV-cache size in bytes. The cache stores one K and one V tensor per layer per token across all L layers, B sequences, context length T_{prev}, and hidden dimension d. Then calculate with $T_{\text{prev}} = 256$.

Deliverable: A few lines of computation.

(b). **[5 pts]** Write the FLOPs for one decode step *without* a cache (re-run the full sequence of length $T_{\text{prev}} + 1$, using your formula from 3.2(a) with $B = 4$) and *with* a cache (process only the one new token against T_{prev} cached keys). Show that the per-step speedup ratio simplifies to T_{prev}.

Deliverable: A few lines of computation.

Section 4: Multi-Layer KV Cache (25 pts)

In production, the KV cache for a large model at long context easily exceeds GPU memory. For example, pythia-160m with a 4096-token context requires $2 \times 12 \times 4096 \times 12 \times 64 \times 2 \approx 144\,\text{MB}$ *per sequence*. Production models are orders of magnitude larger. A real serving system must therefore enforce a global token budget shared across all L layers, evicting old tokens when a layer's quota fills.

Implement the four methods of `MultiLayerKVCache` in `src/kv_cache.py`. The generation functions (`generate_reference`, `generate_with_budget`, `benchmark`) are provided. Model: pythia-160m ($L = 12$, $H = 12$, $d_h = 64$).

4.1: Implement `allocate_uniform` **(4 pts)**

Assign each layer an equal share of the total token budget (`total_budget // num_layers`).

4.2: Implement `allocate_priority` **(5 pts)**

Assign budget proportional to a per-layer importance score. Normalise the scores, multiply by `total_budget`, and floor to integers. The total allocated must not exceed `total_budget`.

For the analysis question you will pass the per-layer rank of the target token from your Section 2 logit-lens results as the importance scores. Higher rank = the layer has not yet converged = more context needed.

4.3: Implement `update` **(5 pts)**

Append new K/V tensors (shape `(1, num_heads, T_new, d_head)`) to the layer's cache by concatenating along the sequence dimension. If the total tokens exceed `layer_budgets[layer_idx]`, evict the oldest tokens from the front (FIFO) and increment `eviction_counts`.

4.4: Implement `get` **(3 pts)**

Return the `(K, V)` tensors for a given layer with shape `(1, num_heads, T_cached, d_head)`, or `(None, None)` if the layer has not yet been written to.

4.5 Analysis (8 pts)

Run the benchmark cells in `hw1.ipynb`. Results are saved automatically to `cache/` and displayed as plots — no manual copying required. Answer the questions below in the notebook.

(a). **[4 pts]** The priority policy gives more budget to layers where the target token has a higher rank. What assumption does this make about the relationship between a layer's prediction uncertainty and its need for long-range context? Is this assumption always valid? Describe a scenario where it could fail even with a generous total budget.

Deliverable: 3–4 sentences.

(b). **[4 pts]** FIFO always evicts the *oldest* tokens first. Propose one alternative eviction criterion, explain what information it uses, and argue why it could preserve generation quality better than FIFO for long-context generation.

Deliverable: 2–3 sentences.